

HASHING

PROBLEM 8

QUESTION:

DS	Session	Hashing	Question Information	Level 1	Challenge 98
Question description You are given an array A of length N which is initialised with 0. You will be given Q queries of two types: 1 k: set value 1 at index k in array A 2 y: print the smallest index x which is greater than or equal to y and having value 1. If there is no such index print -1. Note: Indexing is 1 based Constraints $1 \leq n \leq 10^9$ $1 \leq q \leq 5 * 10^5$ $1 \leq y, k \leq n$ Input Format First line contains two integers N and Q separated by a space. The next Q lines contain the type of query (i.e. either a 1 or a 2), then a space, then for type 1 queries integer k and for type 2 queries integer y Output Format For each query type 2, print in new line, the smallest index x which is greater than or equal to y and having value 1. If there is no such index print -1. Explanation for Test case 1 For first query: $2\ 3$, there is no index greater than or equal index 3, having value -1, so the answer is -1. For second query: $1\ 2$, set value -1 at index 2. For third query: $2\ 1$, index 2 is greater than index 1, having value -1, so the answer is 2. For fourth query: $2\ 3$, there is no index greater than or equal index 3, having value -1, so the answer is -1. For fifth query: $2\ 2$, index 2 is equal to index 2, having value -1, so the answer is 2.					
Logical Test Cases Test Case 1 INPUT (STDIN) 5 5 2 3 1 2 2 1 2 3 2 2 EXPECTED OUTPUT -1 2 -1 2 Test Case 2 INPUT (STDIN) 5 5 2 1 1 2 2 2 1 2 2 3 EXPECTED OUTPUT -1 2 -1					
Mandatory Test Cases Test Case 1 KEYWORD for(i=0;i<q;i++)					
Complexity Test Cases Test Case 1 CYCLOMATIC COMPLEXITY 7 Test Case 2 TOKEN COUNT 225 Test Case 3 NLOC 38					

SOURCE CODE:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct BstItem {
    int val;
    struct BstItem *left, *right;
} BstItem;

BstItem* insert_val(BstItem* root, int key) {
    if (!root) {
        BstItem* p = (BstItem*)malloc(sizeof(BstItem));
        p->val = key;
        p->left = p->right = NULL;
        return p;
    }
    if (key < root->val) root->left = insert_val(root->left, key);
    else if (key > root->val) root->right = insert_val(root->right, key);
    return root;
}

int query_next(BstItem* root, int target) {
    int candidate = -1;
    while (root) {
        if (root->val >= target) {
            candidate = root->val;
            root = root->left;
        } else {
            root = root->right;
        }
    }
    return candidate;
}

int main() {
    int n, queries;
    if (scanf("%d %d", &n, &queries) != 2) return 0;

    BstItem* root = NULL;
    for (int i = 0; i < queries; i++) {
        int action, param;
        scanf("%d %d", &action, &param);
        if (action == 1) {
            root = insert_val(root, param);
        } else {
            printf("%d\n", query_next(root, param));
        }
    }
    return 0;
}
```

OUTPUT:

```
5 5
2 3
1 2
2 1
2 3
2 2
-1
2
-1
2
```

```
5 5
2 1
1 2
2 2
1 2
2 3
-1
2
-1
```